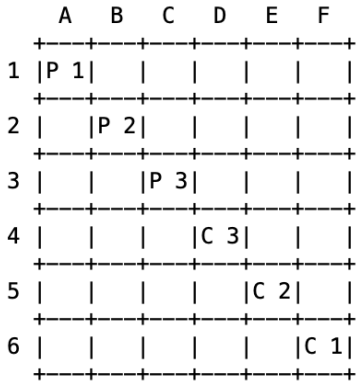
# CS 2340 Term Project Report - Preston Crowe

## *(1) Program Description*

The program is a game called *number chain*, a 6x6 grid game where you chain off your previous spots to get a higher individually valued spot than your opponent. The original starting grids starts as in the picture below:




With 'P' representing the player, 'C' representing the computer, and the value behind it displaying its number value. Whoever goes first is allowed to make any move that surrounds any existing spot they already have, and the number value of that is incremented by one from the highest surrounding of that new spot that player owns. For example, using the screenshot, the player can make a move at C2, which will make the value of that spot '4' since it branches off from C3's '3' that the player owns. However, if the player tries to make a move at F1 for example, since there are no surrounding spots that the player owns around that spot, the move is invalid. Additionally, if the player attempts to make a move at a spot that already has been occupied, the program will notify you of the specific reason why the move is invalid and ask for a different move. This allows the program to guard all use cases for inputs.

The way the input works is by requesting a character A-F and then a number 1-6 to represent the spot. If the format is incorrect, the program will catch it and request a new spot. The program also accommodates for lowercase/uppercase use cases to prevent requesting a new move unnecessarily (by adding/subtracting to the numerical value is the ASCII).

The computer makes a move through the algorithm that will be explained later. Once one player cannot physically make any more valid moves, the computer will recognize that and optimally autofill the rest of the open spots to the other player. After the autofill has been completed/no more moves are able to be placed by either player, the winner check is a simple linear search of each player to find the highest numerical value of each. Once the two values are found, a player win, computer win, or a tie is easily compared with branch statements. In addition, the program randomly selects who goes first by using the random seed syscall and conditional branches to set the game control.

Regarding the files, there are 5 ASM files used: SysCalls.asm for the syscall function .eqv constants, main.asm for the main operations (including music and game control) that call the specific operations, gameboard.asm to control board rendering and lockout reaction, player.asm for the player move and player lockout check, and computer.asm for the computer move and computer lockout check. This system allows for more readable and understandable code as you read through the files.

### *(2) Challenges Faced*

Firstly, since I was trying to create the most optimal algorithm for the computer's decision to move, it took a lot of temporary variables that would be stored in registers. There were multiple instances where I needed more registers but all registers were being used. My solution to this was to move the stack pointer and store the original value of the temporary registers that I knew would not be changed in the time that the program needs more registers, and then move the stack pointer back and restore the original values.

Another huge problem I ran into when programming was the bugs in the complicated algorithms for the lockout check and computer algorithm. My solution to these bugs were debugging with print statements and deep diving in the code and finally finding the small bugs that led to incorrect results in the algorithms. The bug for the computer algorithm was that it was choosing a spot that was not next to a spot the computer owns, and the bug for the lockout check was that computer declared that it was out of moves when there were tons of options for the computer to make in terms of moves (as in, the computer said it was out of moves when it wasn't out of options).

Finally, a different challenge I faced was getting started in general. With so much content and info in front of you before you start the project, knowing where exactly to begin was difficult. My solution to this was mentally planning out in my head how the file distribution would work, creating the files, and starting with the more core idea: the grid itself. I decided on having two separate grids: one to hold the actual grid values and the other to hold which player owns which spots (gridValues and playerGrid).

### *(3) Lessons Learned*

Firstly, I learned that raw memory management with larger programs that require more variables is a huge task and requires a lot of general thinking and sitting at the code rather than actually typing out the lines. During the hefty algorithms, a majority of my time was staring at my own code and figuring out what should go next by imagining everything that is in memory at that current state and how to manage it.

Additionally, I found that I enjoy these types of coding challenges with larger products in difficult settings (such as writing in assembly), as it allowed me to stretch my brain out and design the best algorithms I could for the best game experience while understanding the codebase better every algorithm. As previously mentioned, I had to spend a lot of time simply staring at my code to figure out the state of every register, static data variable, and the stack pointer register. Coding in assembly is definitely much slower than higher level languages.

Finally, I learned that working backwards when designing a game is best. Starting with the core features and eventually ending with game control and winner validation is much easier and more efficient than writing sequentially to the user experience. I began by creating the two arrays, hardcoding the start grid, and printing out the result in a formatted way.

*(Algorithms and Techniques on next page)*

# *Algorithms and Techniques*

## *Algorithm - Player Move / Input Validation*

The player move occurs after the computer has made a move (or if the player starts first) and a lockout check has been made (an algorithm that is explained later in this document). Once the checks have been made, an input syscall is called to request for a 2-character input: an ASCII character and a number. The ASCII character represents the column number A-F (the algorithm will automatically fix the input if it comes in as lowercase). The second character is a number 1-6 to represent the row number on the grid.

The input validation has numerous checks, including incorrect format, out of range, spot has already been occupied, and no valid surroundings. Incorrect format works hand in hand with the spot naturally being out of range such that the program will throw an error to the user and request another input. Once the spot has been declared a valid spot on the grid, a quick check to see if that index of the playerGrid is not zero acts as the use case for if the user requests to occupy a spot is already taken. Lastly, the valid spot will complete a full-circle search around its grid spot and check to see this factor: “are there any surroundings within one spot of this grid spot that the player owns?” If there are no player spots around the input, then the program will throw an error to the user and mention that spot is too far from any player spot to be able to branch off and request a new input.

Once the input has been validated as a correct spot that the player is able to branch off, the computer will calculate the 2-character string mathematically into its correct, word-addressable indexed value and change the grid value to the maximum player surrounding plus one, and the player grid spot to 1 to indicate the player now owns that spot. Once the move has been placed, the program will print “Spot has been taken by player” and print the updated grid.

## *Algorithm - Computer Player AI*

I designed an optimal algorithm for the computer that allows it to make smart decisions while considering the surroundings around it. However, the algorithm does not consider strategy like attempting to block the player or branch off at a lower spot specifically for the long run.

The way the algorithm works for the computer starts with a nested for loop to find the maximum value the computer has in which there are spaces available around it. There are three registers used to store the best spot for the computer to make the move: one for the value of the spot, another for the index, and the last one is for the number of valid surroundings around the spot. The outer loop is the linear search of every spot in the grid. Since the search looks at every spot in the grid, the algorithm does certain checks to skip the inner for loop if not necessary. The inner for loop will only get executed if the computer owns the spot, otherwise the 'i' register increments by four and moves on to the next iteration. In the inner for loop is where the surrounding check occurs and counts the number of valid open spots around the grid space. The inner for loop has 8 iterations to look at every offset possible that surrounds the spot the outer loop has indexed to. For every surrounding that it checks, it will check to see if that surrounding is in range of the grid and is not already taken. Once these checks pass, the counter will increase to track the number of valid surroundings around it.

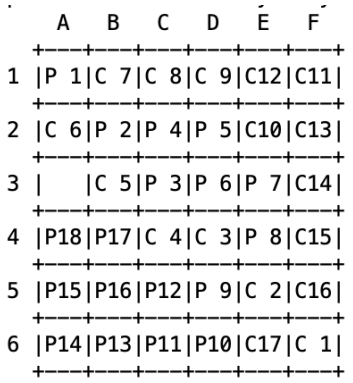
After the nested for loop algorithm is complete, the best grid space with the number of valid surroundings around it is successfully stored in the registers. The next step is to use the random number generator syscall from 0 to the number of valid surroundings in the register (non-inclusive). This allows for all of the valid openings around the best spot to have equal opportunity of being chosen. Once the random number is generated, another for loop goes through the same cycle of surrounding search to choose the random surrounding spot that the computer is going to make. Once the move is made, the program calculates the word-addressable index to the 2-character format and prints that result to the player and sends the updated grid.

```
Computer to E1
     A   B   C   D   E   F
   +---+---+---+---+---+---+
 1 |P 1|C 7|C 8|C 9|C12|C11|
   +---+---+---+---+---+---+
 2 |C 6|P 2|P 4|P 5|C10|   |
   +---+---+---+---+---+---+
 3 |   |C 5|P 3|P 6|P 7|   |
   +---+---+---+---+---+---+
 4 |   |   |C 4|C 3|P 8|   |
   +---+---+---+---+---+---+
 5 |   |   |P12|P 9|C 2|   |
   +---+---+---+---+---+---+
 6 |   |   |P11|P10|   |C 1|
   +---+---+---+---+---+---+
```

*(example of the computer move directly from the program)*

## *Algorithm - Board Rendering*

The board rendering is a simple addition to the game that allows for a better user experience with the GUI.




The screenshot above is a direct example from the program on how the grid is rendered. The algorithm starts by hardcoding the first two ASCII rows with the A-F columns and the '+--+' top row. From there, the rest of the code is a nested for loop, with the outer loop representing the row and the inner loop for each individual column spot within the row. In the outer loop, it starts by calculating the row through integer division and adding 1 to start on the 1-6 scale instead of 0-5. Then, the " |" format string is implemented followed by the inner for loop. Once the inner for loop is done, the hardcoded "+--+" row is hardcoded on the next line.

The inner for loop focuses on the format of each cell. It will first evaluate who owns the spot by looking at the $s1 register and aggregating the index with the word-addressible 'i' value stored in $t0. If the value in the playerGrid at that index is zero, then the algorithm will resort to leaving the grid spot "empty" by adding three ASCII spaces followed by a "|". However, if the value is owned by one of the players, the first character in the call will be a 'C' for the computer or 'P' for the player depending on who owns the spot. Then another check occurs to see if the actual grid value is double digits or not by completing integer division and checking if the result was zero or not. If the value is a double digit integer, then the algorithm will simply print syscall the number followed by a "|" symbol to close the cell. However, if the number is single digit, then the algorithm will print a space followed by the integer value and a "|" symbol to close the cell.

This simple algorithm successfully syscall prints the beautiful grid represented in the screenshot. As you can see, the result accommodates for double digit numbers by maintaining its shape and keeps the sleek format regardless of the grid simulation.

## *Algorithm - Lockout Check -> Autofill -> Winner Check*

This algorithm was the hardest algorithm for me to complete due to the bugs I came across despite the computer move algorithm being a more difficult section to program. After every turn, the program will trigger a lockout check within the individual player.asm and computer.asm files, with the message relaying through a .globl static data variable represented as zero or one. The player lockout check is nearly the exact same as the computer lockout check.

The lockout check works by linearly searching every spot in the array. Depending on if the check is for the human player or the computer, the algorithm will only go into the inner loop if the spot is owned by the player you are looking to check for lockout. Once the outer loop finds a spot that is owned by the player, it will start a while loop that checks the surroundings. If there is a singular valid spot that surrounds the spot, then the whole lockout check will break and the lockout static .globl data will stay as false and continue on in the program. However, if the program traverses through the entire grid without breaking the loop statement, then the algorithm will flip the static data variable that main.asm will check and start the autofill.

The autofill algorithm will traverse linearly through every spot in the grid, stop at the ones that have a grid value of zero, and complete an inner for loop to check the surroundings to get the maximum surrounding around the spot, increment by 1, and insert that as the value. Since the autofill functions exist because one specific player is locked out, then there is no need to look at who owns the surroundings around the grid spaces with a zero on it (the player who was not locked out will surround the spot entirely.

Once autofill is completed, the winner validation check will start. Since the autofill is only called when a player is locked out (which happens every single game), then the winner validation check can be called immediately after autofill is complete. The winner validation check is yet another linear search of the grid, but it will store two temporary variables in registers: one for the maximum grid value for the player and another for the computer. After completing regular comparison checks, the winner check is simply comparing the two registers. If the player's maximum value is higher than the computer's, the winner is the player and vice versa. If they are equal, then it resorts to a tie. A simple SysPrintString syscall is made to send to the console and notify the user who won.

## *Technique/Feature - Audio Playback*

Using my musicianship and knowledge of how to work the audio syscalls in MIPS assembly, I created four separate themes for the game:

1. The main theme: A simple 3-note conjunct melody to an E major chord hold
2. Player Win: A simple 3-note arpeggio step melody to a Bb9 chord and resolve to a Bb major chord.
3. Computer Win: The chromatic 4-note meme melody to make the player feel bad
4. Tie Win: An A 7th full diminished resolved to a Bb minor chord

The main theme is played at the beginning of the program immediately to welcome the player to the game. The last three themes are for the result of the game.